

NineToothed .skill 创新挑战赛 — 中期报告

赛题: 2026 春季 AI 竞赛 — NineToothed .skill 创新挑战赛

赛道编号: T3-1-1

参赛者: Zhao Junkai

GitHub ID: junkai-kay

Skill 名称: ninetoothed-operator-skill

报告时段: 2026-07-03 至 2026-07-11

1. 已完成工作

1.1 仓库调研

- 通读 NineToothed 官方仓库 (README、CONTRIBUTING.md、tests/)
- 分析了官方 test_softmax.py 和 test_add.py 的 API 使用模式
- 输出仓库地图与开发备忘录

1.2 Skill 骨架搭建

- 创建 skills/competition/ninetoothed-operator-skill/ 目录结构
- 编写 SKILL.md (含 YAML 头、9 阶段 SOP、失败诊断 6 步协议)
- 编写 README.md (含四类算子声明、状态表、红线圈)
- 编写 references/index.md (仓库地图、5 种模式、6 项陷阱、代码风格清单)

1.3 四个自测任务设计

任务	类型	算子	文件
T1	逐元素/广播	Masked Add with Broadcast	operator_impl.py + test + benchmark
T2	归约/分块	Tiled Softmax with Numerical Stability	operator_impl.py + test + benchmark

T3	布局敏感	Non-Contiguous GELU	operator_impl.py + test + benchmark
T4	性能/诊断	API 诊断 + Benchmark 矩阵	benchmark_multisize.py + buggy_operator.py + diagnosis_record.md

1.4 GPU 实测结果

环境: Tesla T4, CUDA 12.8, PyTorch 2.11.0, NineToothed 0.26.0

任务	测试结果	Benchmark
T1	12/12 PASSED (122s)	5 配置, 大矩阵反超 PyTorch (0.44x)
T2	13/13 PASSED (8.0s)	6 配置, 多数优于 PyTorch
T3	12/12 PASSED (7.7s)	3 布局, transposed 慢 4.35x
T4	benchmark 就位, 诊断记录完成	8 形状矩阵已完成

1.5 辅助工具

- scripts/env_check.py — 环境依赖检查
- scripts/run_selftests.py — 自动化测试运行器
- tests/test_skill_structure.py — 结构验证 + 占位符检查 + 红线检查

2. 关键技术发现

2.1 NineToothed 0.26 API 实际情况

与网络上的文档不同, 当前版本的重要差异:

- ntl.max() / ntl.sum() 不支持 dim=、axis=、keepdims= 参数, 对整个 tile 归约为标量
- ntl.tanh() 不存在, 需用 exp 手动实现
- 主推 @ninetoothed.jit + Symbol(constexpr=True) 模式
- PyPI 的 ninetoothed 包是空壳, 必须 git clone + pip install -e .

2.2 布局敏感性验证

在同一 4096×4096 矩阵上: – contiguous: NineToothed 与 PyTorch 持平 (1.00x) –

transposed: NineToothed 慢 4.35x（非合并内存访问） – sliced (stride=2): 几乎无影响 (1.01x)

这直接证明了布局影响着 GPU kernel 性能，skill 必须强制测试非连续输入。

3. 遇到的问题与解决

问题	解决
初版代码使用错误 API，T2/T3 全部 CompilationError	Colab 中 <code>dir(ntl)</code> 查看实际 API，参照官方 <code>test_softmax.py</code> 重写
<code>pip install ninetoothed</code> 安装空壳包	改用 <code>git clone + pip install -e .</code>
1D tile 用于 2D tensor 导致编译错误	Tiling 维度必须与 tensor rank 匹配
本地 Mac 无 GPU，无法运行测试	使用 Google Colab 免费 T4 GPU
Git 无法连接 GitHub	配置 SSH key + SOCKS 代理

4. 剩余工作

任务	状态
Proposal 文档	✅ 已完成
中期报告（本文档）	✅ 已完成
T4 子任务 4A（源码导出）和 4C（bug 完整复现记录）	✅ 诊断记录已完成，4A 源码导出待 GPU 环境执行
最终报告 PDF	✅ 已完成
合规审查与 PR 提交	🔄 进行中

5. 个人总结

作为准大二学生，这次比赛是我第一次接触 GPU 算子和 AI 工程化工具链。最大的收获是学会了"以仓库为准"的工作习惯——最初凭直觉和网络搜索写的代码在真实环境中全部编译失败，直到逐行对照官方测试代码才理解正确的 API 模式。这让我意识到在底层开发中，版本

差异和实际 API 远比文档重要。Colab + SSH 的环境搭建、ruff 格式化和 Git hook 的合规流程，也是课堂上学不到的实战经验。虽然过程比预期曲折，但拿到 T1/T2/T3 全部 PASSED 的那一刻，感觉一切努力都值得。